

Nixx Varboncoeur – Homework 06 – MAT320 F25 – 12-06-25

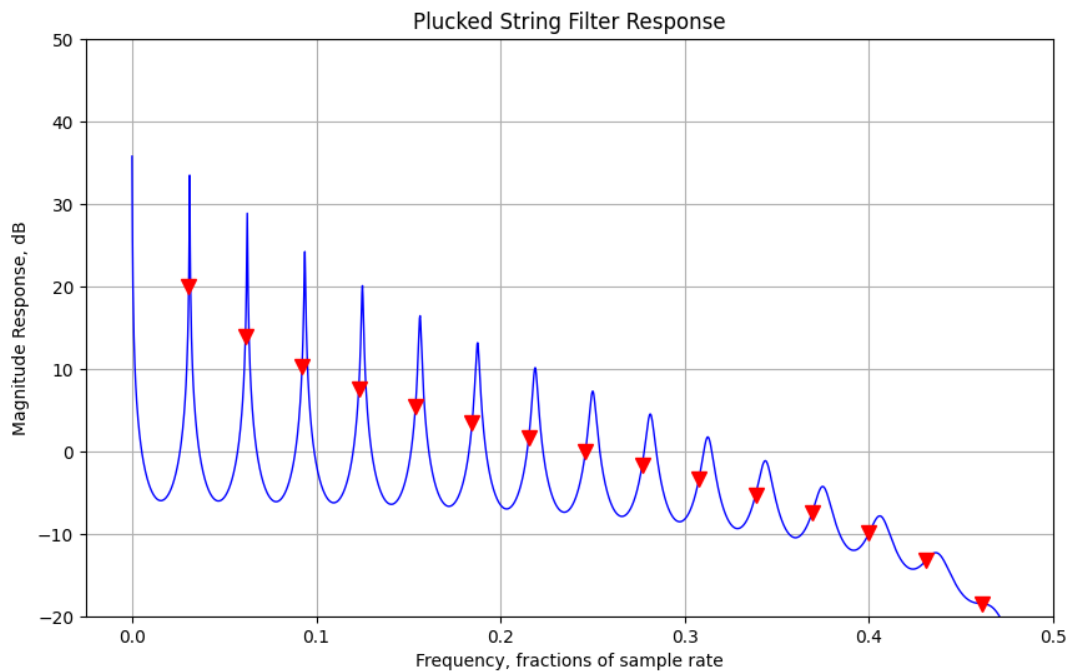
[1]: frequency range between A220 and A1760

This range has 176 distinct pitches

the L-Values in this range are [25, 50, 100, 200]

Frequencies in this range that are in the 5 cent tolerance are [220.5, 441.0, 882.0]

[2a]:



[2b]: calculated tenth harmonic is 0.31266dB.

Actual tenth harmonic is 0.31241dB

This represents an error of 0.08%

[2c] with sample rate 44100:

this discrepancy is -11.03052Hz

[2d] The frequency ratio for this discrepancy is 0.999200

Values > 1 are sharp, < 1 are flat.

[2e] Given that one semitone is 100 cents with formula:

$\text{cent} = [1200 / \ln 2] * [\ln * \text{Frequency Ratio}]$

This discrepancy is a factor of -1.38554 cents

SOURCE CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as sciSig
import os
import sys
from contextlib import redirect_stdout

def calcCentVal(a0, a1):
    '''
    Get x distinct L values in range
    Check how many are within 5 cents of a pitch on
    a even tempered piano
    '''
    sampleRate = 44100
    val = int(a0)
    lVals = []
    while val <= a1:
        lVals.append(int(sampleRate / val))
        val *= 2

    count = len(lVals)
    num = lVals[0] - lVals[count-1]+1
    print(f"[1]: frequency range between A{a0} and A{a1}")
    print("This range has", num, "distinct pitches")
    print("the L-Values in this range are", lVals[::-1])

    cent5Upper = pow(2, (5/1200))
    cent5Lower = pow(2, (-5/1200))

    pianoFreqs = []
```

```

currFreq = a0
while currFreq <= a1:
    pianoFreqs.append(currFreq)
    semitone = pow(2, 1/12) * currFreq
    currFreq = semitone
# print("Piano frequencies in this range are", pianoFreqs)

cent5Tolerance = []
for pianoFreq in pianoFreqs:
    for j in lVals:
        combFreq = sampleRate / j
        freqRatio = combFreq / pianoFreq
        if (cent5Lower <= freqRatio) & (freqRatio <= cent5Upper):
            cent5Tolerance.append(combFreq)
    print("Frequencies in this range that are in the 5 cent tolerance are",
cent5Tolerance, "\n")

def Resonance_PluckedString(L_Val, R_Val, numHarmonix, sampleRate):
    numPoints = 2000
    frequencyArray = np.linspace(0, np.pi, numPoints)
    normalOmega = np.array([omega / (2 * np.pi) for omega in frequencyArray])

    def TransferFunction(freqArray, L, R):
        numerator_r = []
        numerator_i = []
        denom_r = []
        denom_i = []
        for omega in freqArray:
            real_n = np.cos((L + 1)* omega) + np.cos(L*omega)
            numerator_r.append(real_n)

            real_d = 2*np.cos((L + 0.5)* omega) - R*np.cos(omega) - (R ** L)
            denom_r.append(real_d)

            imag_n = np.sin((L+1)* omega) + np.sin(L*omega)
            numerator_i.append(imag_n)

            imag_d = 2*np.sin((L + 0.5) * omega) - R*np.sin(omega)
            denom_i.append(imag_d)
        return numerator_r, numerator_i, denom_r, denom_i
    transferRes = TransferFunction(frequencyArray, L_Val, R_Val)

    def MagnitudeResponse(transferResults):
        '''

```

```

Magnitude = sqrt((real_n2 + imag_n2) / (real_d2 + imag_d2))
'''

num_r = np.array(transferResults[0])
denom_r = np.array(transferResults[2])
num_i = np.array(transferResults[1])
denom_i = np.array(transferResults[3])

numeratorMagnitudes = []
for numR, numI in zip(num_r, num_i):
    num_mag = (numR **2) + (numI **2)
    numeratorMagnitudes.append(num_mag)

denominatorMagnitudes = []
for denomR, denomI in zip(denom_r, denom_i):
    denom_mag = (denomR ** 2) + (denomI **2)
    denominatorMagnitudes.append(denom_mag)

decibels = []
for mag_N, mag_D in zip(numeratorMagnitudes, denominatorMagnitudes):
    result = np.sqrt(mag_N/mag_D)
    result = 20 * np.log10(result)
    decibels.append(result)
return decibels
decibels = np.array(MagnitudeResponse(transferRes))

def ResonanceMarkers(harmonics, lVal):
    """
    Calculate normalized frequency values (x-axis)
    returns: expected omegas in [0, π]
    """
    lRatio = lVal + 0.5
    expected = np.arange(1, harmonics+1) / lRatio
    omegaExpected = []
    for omega in expected:
        # print(omega * 2 * np.pi)
        omegaExpected.append(omega * 2* np.pi)
    return omegaExpected
expectedRads = np.array(ResonanceMarkers(numHarmonix, L_Val))

def ExpectedMagnitude(frequencyArray, decibelArray, expectedOmega):
    markerMag = []
    for omega in expectedOmega:
        closest = np.argmin(np.abs(frequencyArray - omega))
        yVal = decibelArray[closest]

```

```

        markerMag.append(yVal)
    return markerMag

def Plot(omegaArray, normalizedFreq, decibelMag, expectedOmega):
    """
    Plot the frequency response of a plucked string filter
    Args:
        omegaArray: np.array
            Frequency values in the range  $[0, \pi]$ 
        normalizedFreq: list
            Omega values normalized to  $[0, 0.5]$ 
        decibelMag: np.array
            Magnitude response in dB
        expectedOmega: np.array
            Expected calculated resonance frequencies (peaks) in radians
    """
    plt.figure(figsize=(10,6))
    plt.plot(normalizedFreq, decibelMag, 'b-', linewidth=1)
    plt.ylim(-20,50)
    plt.ylabel("Magnitude Response, dB")

    plt.xlim(-0.025, 0.5)
    plt.xlabel("Frequency, fractions of sample rate")

    magOmegas = np.array(ExpectedMagnitude(omegaArray, decibelMag,
expectedOmega))
    # print(magOmegas)
    plt.plot(expectedOmega / (2 * np.pi), magOmegas, 'v', markersize=8,
color='red')

    plt.title("Plucked String Filter Response")
    plt.grid(True)
    os.makedirs('../output', exist_ok=True)
    plt.savefig('../output/pluckedString.png')
    Plot(frequencyArray, normalOmega, decibels, expectedRads)

def HarmonicAnalyze(normalizedFreq, magnitudeDB):
    peaks, properties = sciSig.find_peaks(
        magnitudeDB
        , height=0
        , prominence=10
        , distance=30
    )
    peakFreq = normalizedFreq[peaks]
    peakMag = magnitudeDB[peaks]

```

```

    # harmonicRange = np.arange(1, numHmx+1) / (lval + 0.5)
    expectedTenth = peakFreq[0] * 10
    tenthHmx = peakFreq[9]
    error = 100 * ((tenthHmx - expectedTenth) / expectedTenth)
    return error, tenthHmx, expectedTenth, peakMag

hmxResults = HarmonicAnalyze(normalOmega, decibels)
error = hmxResults[0]
tenthActual = hmxResults[1]
tenthExpected = hmxResults[2]
print(f"[2b]: calculated tenth harmonic is {tenthExpected:.5f}dB.")
print(f"Actual tenth harmonic is {tenthActual:.5f}dB")
print(f"This represents an error of {np.abs(error):.2f}%\n")

discrepancy = tenthActual - tenthExpected
discrepancy = discrepancy * sampleRate
print(f"[2c] with sample rate {sampleRate}:")
print(f" this discrepancy is {discrepancy:.5f}Hz\n")

freqRatio = tenthActual / tenthExpected
print(f"[2d] The frequency ratio for this discrepancy is {freqRatio:.6f}")
print(f"Values > 1 are sharp, < 1 are flat.\n")

cents = (1200 / np.log(2)) * np.log(freqRatio)
print(f"[2e] Given that one semitone is 100 cents with formula:")
print(f"cent = [1200 / ln 2] * [ln * Frequency Ratio]")
print(f"This discrepancy is a factor of {cents:.5f} cents\n")

def main():
    os.makedirs('../output', exist_ok=True)
    with open('../output/results.txt', 'w') as f:
        with redirect_stdout(f):
            calcCentVal(220, 1760)
            Resonance_PluckedString(32, 0.999, 16, 44100)

if __name__ == "__main__":
    main()

```